# Assignment: Concurrent URL Health Checker

**Course:** Systems Programming with Rust
**Estimated Time:** 45 – 60 minutes
**Rust Book Coverage:** Chapters 1 – 17.2

---

## Overview

You will build a command-line tool called `pulse` that concurrently checks the health of a list of URLs, aggregates the results, and prints a structured report to stdout. The focus is on using Rust's ownership model, type system, trait abstractions, smart pointers, and async runtime correctly — not on building a polished user experience.

---

## Learning Objectives

By completing this assignment you will exercise the following Rust concepts in a realistic setting:

- Struct and enum design with associated data
- Custom trait definition and implementation (including `std::fmt::Display`)
- `Result<T, E>` and the `?` operator for error propagation
- Custom error types using enums
- Closures and iterator adapters (`map`, `filter`, `collect`, etc.)
- `Arc<Mutex<T>>` for shared mutable state across threads
- `async` / `await` syntax and `tokio::spawn` for concurrent task execution
- Awaiting multiple futures with `tokio::join!` or `futures::join_all`

---

## Functional Requirements

### FR-1 — URL Input

The program must accept a hard-coded or file-based list of at least **six URLs** to check. At least two of the URLs must be intentionally unreachable or invalid so that your error handling is exercised meaningfully.

You may define the URL list as a static slice in `main.rs` for simplicity.

### FR-2 — Health Check Logic

For each URL, the program must:

1. Send an HTTP GET request.
2. Record the **HTTP status code** if a response was received.
3. Record the **response time** (duration from request start to response received).
4. Classify the result as one of: `Healthy`, `Degraded`, or `Unreachable`.

The classification rules are up to you, but they must be documented in a code comment and must be derived from the status code and/or response time — not hardcoded per URL.

### FR-3 — Concurrency

All HTTP requests must be dispatched **concurrently**, not sequentially. The program must not await each request before starting the next. The total wall-clock time for all checks must be less than `(number of URLs × timeout)`, demonstrating true concurrency.

### FR-4 — Shared Result Collection

Results must be collected into a shared data structure that is safely written to from multiple concurrent tasks. You must use `Arc<Mutex<T>>` for this. You may not use channels as a substitute for this requirement.

### FR-5 — Report Output

After all checks complete, the program must print a report to stdout containing:

- A row per URL showing: URL, status (Healthy / Degraded / Unreachable), HTTP code (or "N/A"), and response time in milliseconds (or "timeout").
- A summary line showing total URLs checked, count of each classification, and total elapsed wall-clock time.

The exact format is your choice, but it must be readable as plain text.

### FR-6 — Timeout

Each individual request must have a configurable timeout (define it as a constant). Requests that exceed the timeout must be classified as `Unreachable` and must not block other results.

---

## Structural Requirements

### SR-1 — `CheckResult` type

Define a struct or enum called `CheckResult` that carries all the information about a single URL check. It must implement `std::fmt::Display`.

### SR-2 — `HealthStatus` enum

Define an enum called `HealthStatus` with at least the three variants listed in FR-2. It must

implement `std::fmt::Display`.

### SR-3 — Custom Error Type

Define a custom error enum (e.g., `PulseError`) that wraps at least two distinct failure modes (e.g., network error, timeout). It must implement `std::fmt::Display` and `std::error::Error`.

Your health-check function's return type must be `Result<CheckResult, PulseError>`.

### SR-4 — Trait: `Reportable`

Define a trait called `Reportable` with at least one method, `summary(&self) -> String`, that produces a one-line human-readable summary of the implementor. Implement it for `CheckResult`.

### SR-5 — Module separation

Your code must be split across at least **two modules** (e.g., `checker` and `report`, or similar). Each module must live in its own file.

---

## Constraints

- You **must** use the `tokio` async runtime (add it to `Cargo.toml` with the `full` feature).
- You **must** use the `reqwest` crate for HTTP (blocking API is not permitted — use the async API).
- You may use any other crates you find useful (e.g., `futures` for `join_all`).
- You **must not** use `unwrap()` or `expect()` anywhere in production code paths. All error cases must be handled through `Result` or graceful fallback logic.
- The program must compile on stable Rust with zero warnings (`cargo build` must succeed cleanly).

---

## Evaluation Criteria

Your submission will be evaluated against the following checklist. Each item is either satisfied or not — there is no partial credit per item.

| # | Criterion |
|---|---|
| 1 | `HealthStatus` enum with `Display` implemented |
| 2 | `CheckResult` struct/enum with `Display` implemented |
| 3 | `PulseError` custom error enum wrapping ⩾ 2 failure modes |
| 4 | Health-check function returns `Result<CheckResult, PulseError>` |
| 5 | `Reportable` trait defined and implemented for `CheckResult` |
| 6 | All requests dispatched concurrently via `tokio::spawn` |
| 7 | `Arc<Mutex<T>>` used for shared result collection |
| 8 | Per-request timeout implemented and classified correctly |
| 9 | Code split across ⩾ 2 modules in separate files |
| 10 | Zero `unwrap()` / `expect()` in non-test code |
| 11 | `cargo build` succeeds with zero warnings |
| 12 | Report output includes per-URL rows and a summary line |

---

## Suggested Development Order

This is a suggestion, not a requirement. You are free to approach the problem in any order.

1. Set up the project and `Cargo.toml` dependencies.
2. Define your types (`HealthStatus`, `CheckResult`, `PulseError`).
3. Implement `Display` and `Reportable` for your types.
4. Write the async health-check function for a single URL.
5. Wire up `tokio::spawn` + `Arc<Mutex<>>` in `main`.
6. Implement the report printer.
7. Verify concurrency by observing total elapsed time.

---

## Deliverable

A single Cargo project directory (`pulse/`) containing all source files, `Cargo.toml`, and a `Cargo.lock`. No write-up required — the code speaks for itself.

*Good luck. When in doubt, let the compiler be your pair programmer.*